UFR de mathématique et d'informatique
Université de Strasbourg

MASTER IN COMPUTER SCIENCE — CODIA TRACK
KNOWLEDGE, DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

Research Study Project (TER)

Quantum Computing for Machine Learning

Network Traffic Classification & Zero-Day Attack Detection

Ashkan MOTAMEDIFAR

Supervised by
Fabrice THÉOLEYRE

Laboratoire ICube – UMR 7357
Université de Strasbourg · CNRS

Strasbourg — May 2026

Contents

1 Introduction **1**

2 Quantum Computing Foundations **2**
2.1 Qubits and Superposition . . . 2
2.2 Multi-Qubit Systems and Entanglement . . . 3
2.3 Quantum Gates . . . 3
2.3.1 Pauli Matrices . . . 3
2.3.2 Rotation Gates . . . 3
2.3.3 Hadamard and Entangling Gates . . . 4
2.4 Measurement . . . 4
2.5 Data Encoding . . . 4
2.5.1 Angle Encoding . . . 4
2.5.2 Dense Encoding (Data Re-uploading) . . . 4
2.6 Parameterized Quantum Circuits and Training . . . 5
2.7 NISQ Constraints . . . 5

3 Quantum Machine Learning Architectures **6**
3.1 Data Re-uploading Classifier . . . 6
3.1.1 Architecture . . . 6
3.1.2 Universality . . . 6
3.1.3 Multi-Qubit Extension . . . 7
3.1.4 Cost Function . . . 7
3.2 Quantum Convolutional Neural Network . . . 7
3.2.1 Architecture . . . 7
3.2.2 Advantages for NISQ Devices . . . 7
3.2.3 Data Encoding for QCNN . . . 8
3.2.4 Cost Function . . . 8
3.3 Hybrid Training Pipeline . . . 8
3.4 Classical Baselines . . . 8

4 Dataset and Preprocessing **9**
4.1 The ToN_IoT Dataset . . . 9
4.2 Feature Selection and Cleaning . . . 9
4.3 Handling Class Imbalance . . . 9
4.4 Normalization and Experimental Splits . . . 10

5 Implementation **11**
5.1 Software Environment . . . 11
5.2 Classical Baselines . . . 11
5.2.1 Support Vector Machine . . . 11
5.2.2 Random Forest . . . 11
5.2.3 Neural Networks . . . 11

5.2.4 Autoencoder (Unsupervised Anomaly Detector) . . . 12
5.3 Quantum Classifiers . . . 12
5.3.1 QCNN . . . 12
5.3.2 Data Re-uploading . . . 13
5.3.3 Training Configuration . . . 13
5.4 Evaluation Protocol . . . 13

6 Results **14**
6.1 Implementation Validation . . . 14
6.2 Standard Classification . . . 14
6.3 Zero-Day Attack Detection . . . 15
6.4 Initialization Sensitivity Analysis . . . 16
6.5 Training Dynamics and Ablation Studies . . . 17
6.5.1 Ablation: Effect of Dataset Size . . . 18
6.6 Discussion . . . 18

7 Conclusion **19**

A Complete Experimental Results **22**
A.1 Confusion Matrices . . . 22
A.2 Initialization Sensitivity . . . 23
A.3 Quantum Circuit Architectures . . . 23

B Bibliography **26**

Chapter 1

Introduction

Machine learning has become a key tool for solving complex classification and prediction tasks across many domains, including network security. Classical models such as neural networks and support vector machines perform well, but their computational cost grows as datasets become larger and more complex. This raises the question of whether alternative computing paradigms could offer practical advantages.

Quantum computing exploits the physical phenomena of superposition and entanglement to process information in fundamentally different ways from classical computers. In recent years, researchers have proposed *parameterized quantum circuits* [1] as trainable models for machine learning, where quantum gate parameters are optimized in the same way as weights in a neural network. Two architectures are particularly relevant to this work: *data re-uploading* [2], which encodes classical data multiple times into a quantum circuit to build a universal classifier, and *Quantum Convolutional Neural Networks* (QCNN) [3], which adapt the classical CNN structure to quantum circuits with logarithmic depth.

These quantum models are designed to operate on current *Noisy Intermediate-Scale Quantum* (NISQ) devices [4], which are limited in both the number of qubits and the circuit depth that can be reliably executed.

In this work, I apply quantum machine learning to **network traffic classification** using the ToN_IoT dataset [5], which contains one million IoT network flow records labeled as normal traffic, Denial-of-Service (DoS) attacks, or injection attacks. The dataset is heavily imbalanced. Because zero-day detection is closer to anomaly detection than to closed-set classification, I additionally include an autoencoder trained on normal traffic only as an unsupervised baseline. I address the following objectives:

1. Compare quantum classifiers with supervised and unsupervised classical baselines on the same network dataset.
2. Evaluate data re-uploading and QCNN architectures in terms of accuracy, parameter efficiency, and initialization sensitivity.
3. Test the ability of these models to detect *zero-day attacks*—attack types absent from the training data.

All quantum experiments are implemented using PennyLane [6]. The remainder of this manuscript covers quantum foundations (Chapters 2–3), the dataset and implementation (Chapters 4–5), results and conclusion (Chapters 6–7).

Chapter 2

Quantum Computing Foundations

This chapter introduces the quantum computing concepts that underpin the machine learning models used in this work. I cover qubits, quantum gates, measurement, data encoding strategies, parameterized quantum circuits, and the constraints of current quantum hardware.

2.1 Qubits and Superposition

In classical computing, the fundamental unit of information is the bit, which takes a value of either 0 or 1. In quantum computing, the analogous unit is the *qubit* [7]. A qubit is a two-level quantum system whose state is a vector in a two-dimensional complex Hilbert space. The two basis states are denoted $|0\rangle$ and $|1\rangle$, and a qubit can exist in a *superposition* of both:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,, \qquad |\alpha|^2 + |\beta|^2 = 1, \tag{2.1}$$

where $\alpha, \beta \in \mathbb{C}$ are complex amplitudes. The values $|\alpha|^2$ and $|\beta|^2$ represent the probabilities of obtaining outcomes 0 and 1 upon measurement.

Since global phase is unobservable, any single-qubit state can be parameterized by two angles $\theta \in [0, \pi]$ and $\varphi \in [0, 2\pi)$:

$$|\psi\rangle = \cos\frac{\theta}{2}\,|0\rangle + e^{i\varphi}\sin\frac{\theta}{2}\,|1\rangle\,. \tag{2.2}$$

This defines a point on the *Bloch sphere* (Figure 2.1), a unit sphere in $\mathbb{R}^3$ where $|0\rangle$ is the north pole, $|1\rangle$ the south pole, and every quantum gate corresponds to a rotation of this sphere.

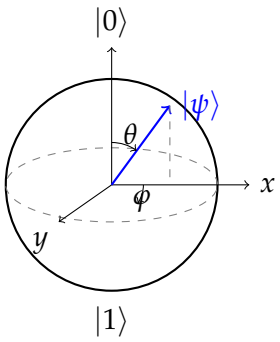


Figure 2.1: The Bloch sphere. Any pure single-qubit state $|\psi\rangle$ corresponds to a point on the surface, parameterized by angles θ and φ.

2.2 Multi-Qubit Systems and Entanglement

The joint state space of n qubits is a 2^n-dimensional Hilbert space. A general pure state can be written as

$$|\Psi\rangle = \sum_{j=0}^{2^n-1} c_j \, |j\rangle \,, \qquad \sum_{j=0}^{2^n-1} |c_j|^2 = 1,$$

where $c_j \in \mathbb{C}$ are complex amplitudes and $\{|j\rangle\}$ denotes the computational basis states.

A state is said to be *entangled* if it cannot be factorized as a tensor product of individual qubit states. A canonical example is the Bell state:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}\big(|00\rangle + |11\rangle \big), \tag{2.3}$$

whose measurement outcomes are perfectly correlated: measuring one qubit determines the outcome of the other.

Entanglement plays a central role in quantum machine learning. Entangling gates applied between qubits encoding different input features allow the circuit to capture correlations that would otherwise require significantly more parameters in a classical model [3].

2.3 Quantum Gates

Quantum gates are unitary transformations that manipulate qubit states. For quantum machine learning, the most important gates are built from the *Pauli matrices* and the Hadamard gate.

2.3.1 Pauli Matrices

The three Pauli matrices form a basis for all single-qubit operations and play a central role throughout quantum computing:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \tag{2.4}$$

Each Pauli matrix is Hermitian and unitary, with $P^2 = I$. In this work they play two direct roles: the rotation gates $R_x(\theta)$, $R_y(\theta)$, $R_z(\theta)$ used as trainable gates throughout are defined as their matrix exponentials (Equation 2.5); and the Pauli-Z matrix serves as the measurement observable whose expectation value $\langle Z \rangle$ is the output of every classifier in this work.

2.3.2 Rotation Gates

The parameterized rotation gates rotate the Bloch vector around the x, y, or z axis by an angle θ:

$$R_x(\theta) = e^{-i\frac{\theta}{2}X}, \qquad R_y(\theta) = e^{-i\frac{\theta}{2}Y}, \qquad R_z(\theta) = e^{-i\frac{\theta}{2}Z}. \tag{2.5}$$

Any single-qubit unitary can be decomposed as $U = R_z(\alpha)\, R_y(\beta)\, R_z(\gamma)$ up to global phase [7]. In quantum machine learning, the angles are either set by the input data (encoding) or treated as trainable parameters (optimization).

2.3.3 Hadamard and Entangling Gates

The **Hadamard gate** $H = \frac{1}{\sqrt{2}}(X + Z)$ maps $|0\rangle$ to the equal superposition $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and is used to initialize qubits in superposition.

For multi-qubit circuits, **entangling gates** create correlations between qubits. The *CNOT* gate flips the target qubit when the control is $|1\rangle$. The *CZ* gate applies a phase flip $|11\rangle \rightarrow -|11\rangle$. Both are used in variational circuits to build correlations between encoded features [8].

2.4 Measurement

Measurement extracts classical information from a quantum state. Measuring $|\Psi\rangle = \sum_j c_j |j\rangle$ in the computational basis yields outcome j with probability $P(j) = |c_j|^2$ (the *Born rule*), after which the state collapses to $|j\rangle$.

In quantum machine learning, the circuit output is typically an *expectation value* of an observable, most commonly the Pauli-Z operator on an output qubit:

$$\langle Z \rangle = P(0) - P(1) \ \in \ [-1, 1]. \tag{2.6}$$

This value serves as the raw prediction of the quantum classifier. Since measurement is probabilistic, the circuit must be executed multiple times (*shots*) to estimate $\langle Z \rangle$ with sufficient precision.

2.5 Data Encoding

A critical step in quantum machine learning is *data encoding* (also called *feature mapping*): the process of embedding classical data $\mathbf{x} \in \mathbb{R}^d$ into a quantum state. The choice of encoding determines how many qubits are required, the circuit depth, and fundamentally shapes what functions the quantum model can learn [9, 10]. Three principal methods exist.

2.5.1 Angle Encoding

In angle encoding, each feature x_i is mapped to a rotation angle on a dedicated qubit [11]:

$$S(\mathbf{x}) = \bigotimes_{i=1}^{d} R_y(x_i) \, |0\rangle \, . \tag{2.7}$$

This requires d qubits (one per feature) but only a single layer of parallel rotations, giving a constant circuit depth of 1. It is the simplest and most widely used method. Its main limitation is that the number of qubits scales linearly with the number of features, so dimensionality reduction must be applied for high-dimensional datasets.

2.5.2 Dense Encoding (Data Re-uploading)

Dense encoding, introduced by Pérez-Salinas et al. [2], takes a fundamentally different approach: instead of dedicating one qubit per feature, it encodes *multiple features into a single qubit* by applying successive parameterized rotations across multiple layers. The data is *re-uploaded*

at each layer, interleaved with trainable parameters. This method uses very few qubits but requires circuit depth proportional to the number of layers L (detailed in Chapter 3).

Table 2.1 summarizes the trade-offs between the two methods used in this work.

Method	**Qubits**	**Depth**	**Features/qubit**
Angle encoding	d	$\mathcal{O}(1)$	1
Dense (re-uploading)	1	$\mathcal{O}(L)$	all per layer

Table 2.1: Comparison of data encoding methods. d: number of features, L: number of re-uploading layers.

In this work, I use angle encoding for the QCNN and dense encoding for the data re-uploading classifier.

2.6 Parameterized Quantum Circuits and Training

A *parameterized quantum circuit* (PQC) is the quantum analogue of a neural network [1]. It consists of three stages: (1) data encoding, (2) a sequence of trainable rotation gates interleaved with entangling gates, and (3) measurement. The output defines a function

$$f(\mathbf{x};\boldsymbol{\theta}) = \langle 0^n | \, U^\dagger(\mathbf{x},\boldsymbol{\theta}) \, Z \, U(\mathbf{x},\boldsymbol{\theta}) \, |0^n\rangle \,,$$

where $\boldsymbol{\theta}$ are the trainable parameters.

To optimize $\boldsymbol{\theta}$, gradients are computed using the *parameter-shift rule* [12]: for each parameter θ_k appearing in a rotation gate, the partial derivative is:

$$\frac{\partial f}{\partial \theta_k} = \frac{1}{2}\Big[f\big(\theta_k + \tfrac{\pi}{2}\big) - f\big(\theta_k - \tfrac{\pi}{2}\big)\Big]. \tag{2.8}$$

This provides exact gradients without finite-difference approximations; PennyLane [6] implements it automatically.

2.7 NISQ Constraints

Current quantum computers are Noisy Intermediate-Scale Quantum (NISQ) devices [4], characterized by three limitations: (1) a small number of qubits (tens to hundreds), limiting the data dimensionality that can be directly encoded; (2) gate errors and *decoherence*, which impose an upper bound on circuit depth before outputs become unreliable; and (3) limited qubit connectivity, requiring extra SWAP gates for non-adjacent interactions.

These constraints motivate shallow-depth architectures. The data re-uploading approach [2] achieves universality with linear depth using only one or a few qubits. The QCNN [3] achieves logarithmic depth through hierarchical pooling and has been shown to avoid the *barren plateau* problem [13], where gradients vanish exponentially with circuit size.

In this work, all experiments use noiseless simulation via PennyLane, but the circuit designs are chosen to be compatible with near-term hardware: shallow depth, nearest-neighbor connectivity, and a small number of qubits.

Chapter 3

Quantum Machine Learning Architectures

This chapter describes the two quantum classification architectures investigated in this work: the data re-uploading classifier [2] and the Quantum Convolutional Neural Network (QCNN) [3]. For each architecture, I present the circuit structure, the rationale behind its design, and the cost functions used for training. I then describe the classical baselines used for comparison.

3.1 Data Re-uploading Classifier

The data re-uploading classifier, proposed by Pérez-Salinas et al. [2], is based on a key insight: a single qubit can act as a universal classifier if the classical input data is *re-uploaded* into the circuit multiple times, interleaved with trainable rotations.

3.1.1 Architecture

The circuit consists of L layers, each applying a general SU(2) rotation to a single qubit. In each layer l, the rotation angles are a linear combination of the input data $\mathbf{x}$ and trainable parameters:

$$\mathcal{L}^{(l)}(\mathbf{x}, \boldsymbol{\theta}^{(l)}, \mathbf{w}^{(l)}) = R_z\big(\theta_3^{(l)} + w_3^{(l)} x_3\big)\; R_y\big(\theta_2^{(l)} + w_2^{(l)} x_2\big)\; R_z\big(\theta_1^{(l)} + w_1^{(l)} x_1\big), \tag{3.1}$$

where $\boldsymbol{\theta}^{(l)}$ are bias parameters and $\mathbf{w}^{(l)}$ are weight parameters that control how strongly each feature influences the rotation. The full circuit applies all layers sequentially to the initial state $|0\rangle$:

$$|\psi(\mathbf{x})\rangle = \mathcal{L}^{(L)} \cdots \mathcal{L}^{(2)}\, \mathcal{L}^{(1)}\, |0\rangle\,. \tag{3.2}$$

Each layer has 6 trainable parameters (3 biases and 3 weights), so an L-layer single-qubit circuit has $6L$ parameters in total.

3.1.2 Universality

A central result of Pérez-Salinas et al. [2] is that the data re-uploading circuit is a *universal quantum classifier*: with enough layers, it can approximate any continuous classification boundary. Each additional layer increases the degree of the trigonometric polynomial that the circuit can represent [10], playing the same role as depth in a classical network. In practice, $L = 5$–20 layers suffice for common tasks.

3.1.3 Multi-Qubit Extension

When more qubits are available, the data re-uploading scheme extends naturally: each qubit receives its own re-uploading layers processing a subset of the input features, and entangling gates (CZ) are applied between neighboring qubits after each layer to create correlations. This allows the circuit to capture relationships between features encoded on different qubits and reduces the number of layers required to achieve a given accuracy.

In the multi-qubit variant, each qubit independently applies R_z–R_y–R_z rotations mixing data and trainable weights, then CZ gates entangle neighboring qubits before the next layer begins. For n qubits with L layers, the total parameter count is $6nL$ (the CZ gates are parameter-free). The full circuit diagram of the 4-qubit, 3-layer variant used in this work is given in Figure A.4 of Appendix A.3.

3.1.4 Cost Function

Pérez-Salinas et al. [2] proposed fidelity-based cost functions that measure the overlap between the circuit output state and a target state assigned to each class. In this work, I use MSE loss with binary targets $\{-1, +1\}$ for consistency with the QCNN (Section 3.2.4).

3.2 Quantum Convolutional Neural Network

The Quantum Convolutional Neural Network (QCNN), proposed by Hur et al. [3], adapts the hierarchical structure of classical CNNs to quantum circuits. It consists of alternating convolutional and pooling layers that progressively reduce the number of active qubits until a final measurement is performed.

3.2.1 Architecture

The QCNN operates on n qubits (typically $n = 2^k$) and consists of three types of layers:

Convolutional layers apply the same parameterized two-qubit unitary $U(\boldsymbol{\theta})$ to all neighboring pairs of qubits. The weight sharing (same parameters for all pairs) mirrors the translational invariance of classical convolutional filters and significantly reduces the parameter count. Hur et al. [3] tested 9 different circuit ansätze for this unitary, varying in the number of CNOT and rotation gates. In this work, I use a 6-parameter R_y / R_z + CNOT variant inspired by their family of ansätze; the exact form and the resulting parameter count are given in Section 5.3.

Pooling layers reduce the qubit count by half. A controlled rotation is applied to each pair, then one qubit is traced out (measured and discarded). This progressive halving gives the QCNN its $\mathcal{O}(\log n)$ depth.

Fully connected layer. After repeated stages, a final $R_z R_y R_z$ rotation is applied to the remaining qubit before measuring $\langle Z \rangle$. The full circuit diagram of the 8-qubit QCNN used in this work is given in Figure A.2 of Appendix A.3.

3.2.2 Advantages for NISQ Devices

The QCNN has two properties that make it particularly suitable for near-term quantum hardware. First, its $\mathcal{O}(\log n)$ depth makes it inherently shallow: for 8 qubits, only 3 stages are needed. Second, Hur et al. [3] demonstrated that the hierarchical pooling structure avoids the

barren plateau problem [13], a phenomenon where gradients of the cost function vanish exponentially with circuit size in generic variational circuits, making training practically impossible. The QCNN's local structure and progressive qubit reduction maintain trainable gradients even as the system grows.

3.2.3 Data Encoding for QCNN

Since the QCNN requires one qubit per input feature, the input dimension must match the qubit count. For datasets with more features than qubits, dimensionality reduction is applied beforehand. Hur et al. [3] tested several encoding strategies, including angle encoding (R_y rotations), dense encoding ($R_y R_z$ per qubit), and hybrid approaches. In this work, I use angle encoding for its simplicity and direct compatibility with the 8-qubit circuit.

3.2.4 Cost Function

The circuit output $f(\mathbf{x}; \boldsymbol{\theta}) = \langle Z \rangle \in [-1, +1]$ is trained with MSE loss:

$$C_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} \big(y_i - f(\mathbf{x}_i; \boldsymbol{\theta})\big)^2, \tag{3.3}$$

where $y_i \in \{-1, +1\}$ is the binary target. Hur et al. [3] also explored cross-entropy, which maps $\langle Z \rangle$ to a probability via $p_i = \frac{1}{2}(1 + \langle Z \rangle)$ and performs slightly better; I use MSE for simplicity.

3.3 Hybrid Training Pipeline

Both architectures follow the standard *hybrid quantum-classical* training loop [8]: the quantum circuit evaluates $f(\mathbf{x}_i; \boldsymbol{\theta})$ for each sample, gradients are computed on the classical side via the parameter-shift rule (Equation 2.8), and parameters are updated with Adam [14]. The quantum device handles only circuit evaluation; all optimization runs classically, making the approach feasible on NISQ hardware.

3.4 Classical Baselines

To assess whether quantum models offer any practical advantage, I compare them against an SVM (RBF kernel), a Random Forest, and two small neural networks whose parameter counts are matched to those of the quantum circuits. This parameter-matched methodology follows Hur et al. [3]. Full implementation details and results are given in Chapters 5 and 6.

Chapter 4

Dataset and Preprocessing

This chapter describes the dataset, the preprocessing pipeline, and the experimental splits used for both standard classification and zero-day attack detection.

4.1 The ToN_IoT Dataset

I use the ToN_IoT dataset [5], captured from an IoT testbed at UNSW. Specifically, I work with `Network_dataset_11`: 1,000,000 flow records over 46 columns combining transport-layer metadata (ports, protocol, duration, byte and packet counts), application-layer fields (DNS, HTTP, SSL), and a traffic class label.

The dataset contains three classes: *normal* traffic (35,168 samples, 3.5%), *DoS* attacks (839,637 samples, 84.0%), and *injection* attacks (125,195 samples, 12.5%).

4.2 Feature Selection and Cleaning

Of the 46 original columns, 21 are constant across all records (SSL, HTTP, and miscellaneous fields uniformly `'-'` or 0) and carry no information. I also drop capture-specific identifiers (`ts`, `src_ip`, `dst_ip`) that would not generalize, the text-valued `dns_query`, and the redundant binary `label`. This leaves 19 features (see Table A.1 in Appendix A).

To reduce dimensionality for quantum encoding, I first perform the train/test split (Section 4.4), then—using the training set only, to avoid data leakage [15]—compute pairwise Pearson correlations and drop features with correlation above 0.95, rank the rest by ANOVA F-score, and keep the top 8 to match the qubit count. The same features are then applied to the test set without refitting. Figure 4.1 shows the ranking; the selected features are `src_port`, `service`, `dst_port`, `conn_state`, `src_pkts`, `src_ip_bytes`, `proto`, and `dns_AA`.

4.3 Handling Class Imbalance

The severe class imbalance is problematic for training. Interpolation-based augmentation (e.g., SMOTE) is unsuitable here: a linear mix of two flows with different TCP flags or connection states produces a flow that cannot exist in practice. I instead use *random undersampling* of the majority classes to match the minority (35,168 per class), giving a balanced dataset of 105,504 samples; quantum experiments subsample further due to simulation cost.

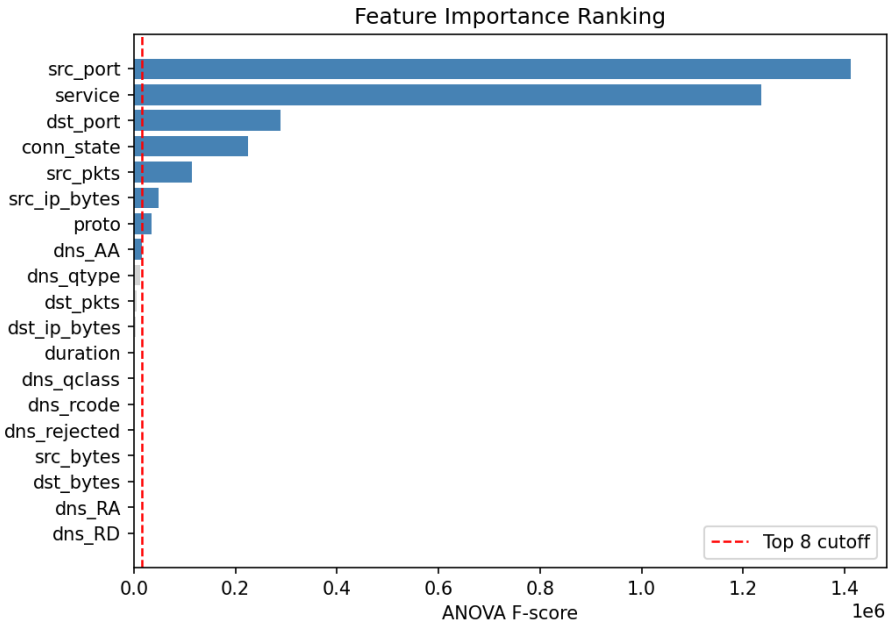


Figure 4.1: ANOVA F-score ranking of the 19 candidate features. The top 8 (blue) are selected for quantum encoding; the red dashed line marks the cutoff.

4.4 Normalization and Experimental Splits

For angle encoding (Section 3.2), each feature is scaled to $[0, \pi]$ via min-max normalization:

$$x_i^{\text{scaled}} = \pi \cdot \frac{x_i - \min x_i}{\max x_i - \min x_i}, \tag{4.1}$$

so each feature spans the full range of a rotation gate. The scaler is fit on the training set only.

I define two experimental settings:

Standard classification. The balanced dataset is split 80/20 with stratified sampling (84,403 training / 21,101 test). All models share the same splits.

Zero-day attack detection. The model is trained on normal and DoS traffic only, then evaluated on injection attacks absent from training—simulating an unseen attack type (Figure 4.2).

For quantum experiments, the standard 3-class split is stratified-subsampled to 500 training samples and 200 test samples, since circuit simulation on classical hardware scales linearly with dataset size. The zero-day quantum split uses fixed (non-stratified) random subsamples of the same size from the normal+DoS training set and the injection-only test set.

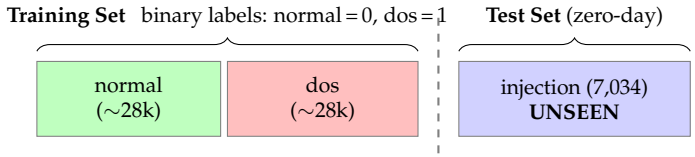


Figure 4.2: Zero-day experimental split. The model trains on normal and DoS traffic only (binary classification), then is tested on injection attacks — an entirely unseen class. High detection rate indicates genuine generalization beyond memorized attack patterns.

Chapter 5

Implementation

This chapter describes the implementation of all classifiers evaluated in this work. The full source code is publicly available [16] and organised in the `src/` directory, written in Python 3.11.

5.1 Software Environment

Three main libraries are used. **PennyLane** [6] (v0.39) provides the QNode interface for circuit evaluation, the `lightning.qubit` noiseless simulator used in the main experiments, and the `AdamOptimizer`; gradients are computed with adjoint differentiation for speed. **scikit-learn** [17] provides the classical classifiers and model-selection utilities. **pandas** and **NumPy** handle data loading and numerical operations. All experiments run on CPU under noiseless simulation; no real quantum hardware is used.

5.2 Classical Baselines

The classical baselines are implemented in `src/classical/classical_baselines.py` and evaluated under both experimental settings defined in Section 4.4.

5.2.1 Support Vector Machine

The SVM uses an RBF kernel. Hyperparameters $C \in \{0.1, 1, 10, 100\}$ and $\gamma \in \{\texttt{scale}, \texttt{auto}, 0.01, 0.1\}$ are selected by 5-fold cross-validation on the training set. The best configuration is $C = 100$, $\gamma = \texttt{scale}$, achieving a cross-validation accuracy of 97.8%.

5.2.2 Random Forest

A Random Forest of 100 decision trees is trained with default scikit-learn settings. As an ensemble method, it provides a robust baseline for tabular data and additionally yields feature importance scores that complement the ANOVA-based selection in the preprocessing stage.

5.2.3 Neural Networks

Two fully connected networks with ReLU activations are trained using Adam ($\eta = 10^{-3}$, up to 200 epochs):

- **NN-Small** ($8 \to 12 \to 6 \to 3$, $\sim$207 parameters): a small network in the same order of magnitude as the quantum classifiers.
- **NN-Medium** ($8 \to 16 \to 8 \to 3$, $\sim$307 parameters): a slightly larger network to test whether additional classical capacity helps.

5.2.4 Autoencoder (Unsupervised Anomaly Detector)

Supervised classifiers learn the boundary between the classes they are shown during training, which is precisely why they fail on unseen attack types. As a fairer classical baseline for the zero-day setting I add a shallow *autoencoder* trained on normal traffic only. The architecture is a symmetric $8 \to 6 \to 3 \to 6 \to 8$ MLP with tanh activations and Xavier initialization, optimized with Adam ($\eta = 2 \times 10^{-3}$, weight decay 10^{-5}, cosine learning-rate schedule) over 100 epochs on the $\sim$28,000 normal training samples. At inference, an input is flagged as anomalous if its reconstruction MSE exceeds the 95th percentile of the training reconstruction errors. The threshold is set to the 95th percentile of the training-normal reconstruction errors; the held-out normal split is used only to estimate the resulting false-positive rate, which is approximately 5%. This protocol turns "zero-day detection" into "anything that does not look like normal traffic," and is the standard unsupervised approach to network intrusion detection.

Table 5.1 gives an overview of all models and their parameter budgets.

Model	Type	Parameters
SVM (RBF, C=100)	Classical	kernel-based
Random Forest (100 trees)	Classical	tree-based
NN-Small $(12, 6)$	Classical	$\sim$207
NN-Medium $(16, 8)$	Classical	$\sim$307
Autoencoder $(8{-}6{-}3{-}6{-}8)$	Classical	$\sim$160
Data Re-uploading (1 qubit, 6 layers)	Quantum	108
Data Re-uploading (4 qubits, 3 layers)	Quantum	216
QCNN (8 qubits, 3 stages)	Quantum	81

Table 5.1: Parameter counts for all evaluated models. Quantum counts are the total for the 3-class one-vs-all system (3$\times$ the per-classifier count).

5.3 Quantum Classifiers

Both quantum classifiers are implemented in `src/quantum/` using PennyLane's `lightning.qubit` simulator with adjoint differentiation, which gives identical results to `default.qubit` but runs noticeably faster. They share the same one-vs-all strategy: for $K = 3$ classes, three independent binary circuits are trained, each targeting one class with labels $y_i \in \{-1, +1\}$. At inference, the class whose circuit returns the highest $\langle Z \rangle$ is predicted.

5.3.1 QCNN

The QCNN operates on 8 qubits with 3 conv-pool stages ($8 \to 4 \to 2 \to 1$), followed by an $R_z R_y R_z$ fully connected layer on the remaining qubit. The convolutional unitary is a 6-parameter two-qubit block inspired by the ansatz family of Hur et al. [3]:

$$U(\boldsymbol{\theta}) = \left[R_y R_z\right]_{q_1} \text{CNOT} \left[R_y R_z\right]_{q_0} \left[R_y R_z\right]_{q_1},$$

a lighter variant of Hur et al.'s 10-parameter U_6 (which uses R_x/R_z rotations and CRX entanglement). Each pooling layer uses a CRZ–CRY pair (2 parameters) to conditionally transfer information before discarding one qubit, a simplification of the CRZ + controlled-R_x pooling proposed in the original paper. The total per-classifier parameter count is $3 \times 6 + 3 \times 2 + 3 = 27$, and 81 parameters for the full 3-class system.

Input data is encoded via angle encoding: each of the 8 features is mapped to an R_y rotation on the corresponding qubit.

5.3.2 Data Re-uploading

Two variants are implemented:

- **Single-qubit**: 6 layers on 1 qubit, $6 \times 6 = 36$ parameters per classifier (108 total).
- **Multi-qubit**: 3 layers on 4 qubits with nearest-neighbor CZ entanglement, $6 \times 4 \times 3 = 72$ parameters per classifier (216 total).

In each layer, rotation angles are computed as $\theta_j^{(l)} + w_j^{(l)} x_{f(j,l)}$, where the feature index $f(j,l)$ cycles through the input features across layers. For the multi-qubit variant, the output is the average $\langle Z \rangle$ over all qubits, expressed as a single Hamiltonian measurement $\langle H \rangle$ with $H = \frac{1}{n} \sum_q Z_q$.

5.3.3 Training Configuration

All quantum classifiers are trained with the Adam optimizer (learning rate $\eta = 0.01$) for 100 epochs. Each one-vs-all binary classifier uses a *separate* optimizer instance to avoid momentum interference between sub-models. Gradients are computed via adjoint differentiation, which is mathematically equivalent to the parameter-shift rule but considerably faster in simulation. Parameters are initialized uniformly in $[-0.1, 0.1]$ with a controlled random seed to ensure reproducibility and to mitigate barren plateaus [13]. To account for the known sensitivity of quantum circuits to initialization, the main experiments are repeated with 10 different seeds and results are reported as mean $\pm$ standard deviation (Section 6.4). For the standard 3-class task, training uses a stratified subsample of 500 samples and test evaluation uses 200 stratified samples; for zero-day evaluation, the same sizes are drawn as fixed (non-stratified) random subsamples of the normal+DoS train set and the injection-only test set. All circuits run on noiseless simulation.

5.4 Evaluation Protocol

All models are evaluated under the two settings defined in Section 4.4:

- **Standard**: 3-class accuracy and weighted F1-score on the 200-sample test set.
- **Zero-day**: binary detection rate on the injection-only test set (7,034 samples after the 80/20 split), where the model has only seen normal and DoS traffic during training. Quantum models are evaluated on a fixed 200-sample subsample of this set due to simulation cost.

Results are reported in Chapter 6.

Chapter 6

Results

This chapter begins with a validation experiment that confirms the correctness of my quantum implementation (Section 6.1), then presents the main results for standard classification (Section 6.2) and zero-day attack detection (Section 6.3). Section 6.4 analyzes parameter initialization sensitivity, Section 6.5 discusses training dynamics and ablation studies, and Section 6.6 summarizes the findings. All quantum results are averaged over 10 random seeds; the autoencoder is also evaluated on 10 seeds. A consolidated table of all results is provided in Appendix A.

6.1 Implementation Validation

Before evaluating on the IoT dataset, I validate the data re-uploading implementation on two standard synthetic benchmarks from the literature. Pérez-Salinas et al. [2] report 90–98% accuracy on similar 2D classification tasks. I train a single-qubit re-uploading circuit (4 layers, 24 parameters) for 80 epochs on 300 training samples from `make_circles` and `make_moons` (scikit-learn). Table 6.1 reports the results.

Dataset	Params	Accuracy	F1	MCC
`make_circles`	24	0.990	0.990	0.980
`make_moons`	24	1.000	1.000	1.000

Table 6.1: Validation results on synthetic 2D datasets. My implementation achieves 99–100% accuracy, consistent with the 90–98% range reported by Pérez-Salinas et al. [2], confirming that the quantum circuit and training pipeline are correctly implemented.

6.2 Standard Classification

Table 6.2 reports accuracy, weighted F1-score, and Matthews Correlation Coefficient (MCC) on the 200-sample test set for the standard 3-class setting. MCC is included as it provides a balanced measure even when class sizes differ slightly [18]. Quantum results are reported as mean $\pm$ standard deviation over 10 random seeds (see Section 6.4).

All classical models achieve high accuracy ($\geq$ 92.5%), indicating strong class separation in the selected features. The quantum models perform significantly worse: the best quantum result is

Model	Type	Accuracy	F1	MCC
Random Forest	Classical	0.990	0.990	0.985
SVM (RBF)	Classical	0.970	0.970	0.955
NN-Medium (16, 8)	Classical	0.960	0.960	0.940
NN-Small (12, 6)	Classical	0.925	0.925	0.888
Re-uploading 1q	Quantum	0.752 ± 0.039	0.720 ± 0.051	0.662 ± 0.045
Re-uploading 4q	Quantum	0.692 ± 0.011	0.654 ± 0.010	0.586 ± 0.016
QCNN (8q)	Quantum	0.679 ± 0.025	0.636 ± 0.021	0.559 ± 0.043

Table 6.2: Standard 3-class results (normal / DoS / injection). Classical models are trained on 500 quantum-subset samples and tested on 200 samples. Quantum results are mean $\pm$ std over 10 seeds.

the single-qubit re-uploading at 75.2% accuracy (MCC 0.66). The QCNN achieves 67.9% with an MCC of 0.56.

These classical results are consistent with prior evaluations of ML models on ToN_IoT: Moustafa [19] reports that tree-based and kernel methods consistently achieve >95% accuracy on this dataset, confirming that our preprocessing pipeline and baseline implementation are correct.

This gap between classical and quantum is consistent with prior work showing that quantum classifiers do not outperform classical methods on datasets that are already linearly or near-linearly separable in the original feature space [10, 20]. With 8 well-chosen features, a Random Forest achieves 99% accuracy with only 500 training samples, leaving no room for quantum advantage on this task.

6.3 Zero-Day Attack Detection

Table 6.3 reports results on the zero-day setting, where models are trained exclusively on normal and DoS traffic and then tested on injection attacks—a class entirely absent from training. Since the test set is homogeneous (all injection), I report accuracy as the primary metric (equivalent to recall for the attack class). Quantum and autoencoder results are mean $\pm$ std over 10 seeds.

The results separate the seven evaluated models into three regimes:

Supervised classical models fail completely. Every supervised classical baseline (Random Forest, SVM, NN-Small, NN-Medium) detects fewer than 3% of injection attacks. This is the expected behaviour: a classifier trained to discriminate *normal* from *DoS* learns a boundary tuned to those two distributions, and an unseen attack class lying away from that boundary is silently absorbed into one of the known classes.

Two approaches reach $\sim$70%. The autoencoder (70.5% $\pm$ 14.4%) and the QCNN excluding barren-plateau seeds (68.6% $\pm$ 6.8%) reach essentially the same mean detection rate, an order of magnitude above the supervised classical baselines—but they are not both unsupervised. The autoencoder is trained on normal traffic only and flags any input that does not reconstruct well (genuine unsupervised anomaly detection). The QCNN is trained *supervised* on normal-vs-DoS labels and happens to transfer to injection on 7 out of 10 initializations, suggesting that its quantum feature map captures a broader notion of "not normal" rather than memorizing the DoS distribution. Both approaches share the underlying intuition of modelling normal traffic and treating anything anomalous as an attack, but reach it through fundamentally different

Model	Type	Accuracy	F1
Autoencoder (anomaly detection)	Classical	**0.705 ± 0.144**	**0.819 ± 0.100**
QCNN (8q), excl. barren-plateau seeds	Quantum	**0.686 ± 0.068**	**0.812 ± 0.051**
QCNN (8q), all 10 seeds	Quantum	0.490 ± 0.305	0.587 ± 0.349
Re-uploading 1q	Quantum	0.092 ± 0.036	0.167 ± 0.058
Re-uploading 4q	Quantum	0.075 ± 0.023	0.138 ± 0.039
NN-Small (12, 6)	Classical	0.028	0.054
SVM (RBF)	Classical	0.024	0.046
NN-Medium (16, 8)	Classical	0.017	0.034
Random Forest	Classical	0.003	0.007

Table 6.3: Zero-day detection results. Models are trained on normal + DoS only; the test set contains exclusively injection attacks (unseen during training). Supervised classical models are evaluated on the full zero-day test set (7,034 samples); the autoencoder uses the same 7,034 samples as test set and is trained on all ∼28,000 normal samples; quantum models use a 200-sample test subsample and 500 training samples due to simulation cost. Autoencoder and quantum results are mean ± std over 10 seeds. The "excl. barren-plateau seeds" row reports the mean over the 7 seeds for which the QCNN converges; the 3 collapsed seeds $\{2, 5, 6\}$ (zero-day detection $\leq 9\%$) are excluded. Per-seed values are given in Table 6.4 (Section 6.4).

training regimes.

Re-uploading remains brittle on zero-day. Both re-uploading variants stay at 7–9% on injection (versus ∼2–3% for supervised classical), indicating they generalize slightly beyond their training classes but fall well short of the QCNN and the autoencoder. Their flatter feature maps appear to fit the specific DoS distribution rather than learning a broader notion of "normal" versus "attack".

The QCNN's headline number depends critically on whether barren-plateau seeds are included: across all 10 seeds the mean is 49.0% with a 30.5% standard deviation, masking a bimodal behaviour with 7 successful runs (54–76%) and 3 training collapses ($\leq 9\%$). Section 6.4 returns to this point. The autoencoder is more stable across seeds in terms of failure rate (1/10 vs. 3/10), but its detection rate when training succeeds varies more (std 14.4% vs. 6.8% for the QCNN).

A methodological note. The autoencoder is trained on all ∼28,000 normal samples while the quantum classifiers are restricted to the 500-sample quantum subsample for simulation-cost reasons (Chapter 4). The two methods therefore see different amounts of data, and this asymmetry should temper any direct numerical comparison. It does not, however, change the qualitative ranking observed: supervised classical $\ll$ re-uploading $<$ {autoencoder, QCNN}.

6.4 Initialization Sensitivity Analysis

The large standard deviation observed in QCNN zero-day performance motivated a systematic analysis of parameter initialization sensitivity. Table 6.4 shows the per-seed QCNN results on the zero-day task (500 training samples, 100 epochs, 10 seeds).

Two observations emerge. **Standard classification is stable** across seeds (coefficient of variation 3.7%); **zero-day detection is bimodal**, with 7/10 seeds reaching 54–76% and 3 collapsing to $\leq$ 9%. This is consistent with the barren plateau phenomenon [13]: gradient magnitudes vanish exponentially with circuit width, leaving the optimizer stuck in a flat region. The 30%

Seed	Std. Acc.	Std. F1	ZD Acc.	ZD F1
0	0.720	0.674	0.755	0.860
1	0.675	0.638	0.640	0.780
2	0.625	0.593	*0.010*	*0.020*
3	0.685	0.616	0.730	0.844
4	0.680	0.641	0.695	0.820
5	0.690	0.650	*0.000*	*0.000*
6	0.665	0.629	*0.090*	*0.165*
7	0.680	0.642	0.710	0.830
8	0.660	0.625	0.730	0.844
9	0.710	0.651	0.540	0.701
Mean $\pm$ std (all 10)	0.679 ± 0.025	0.636 ± 0.021	0.490 ± 0.305	0.587 ± 0.349
Mean (excl. barren: 2,5,6)	0.687 ± 0.019	0.641 ± 0.017	0.686 ± 0.068	0.812 ± 0.051

Table 6.4: Per-seed QCNN results (500 training samples, 100 epochs, 10 seeds). Standard accuracy is stable across seeds (66–72%), but zero-day performance is bimodal: 7 seeds achieve 54–76%, while 3 seeds (italics) collapse to $\leq$ 9%. The collapses are consistent with the barren plateau phenomenon [13].

failure rate is not negligible—a single-seed evaluation could have produced either a strong false claim of quantum advantage or of total failure. Multi-seed evaluation is therefore not optional for variational quantum models. The re-uploading classifiers, by contrast, have a smoother landscape (coefficient of variation $\sim$30% on zero-day, $\sim$5% on standard) but also fail to generalize to the unseen class. The autoencoder has its own failure mode (1/10 seeds: near-identity reconstruction, detectable from a DoS sanity check), but at a meaningfully lower rate (10% vs. 30%).

6.5 Training Dynamics and Ablation Studies

Figure 6.1 shows the training loss (MSE) over 100 epochs for the quantum models on the standard setting (seed 0).

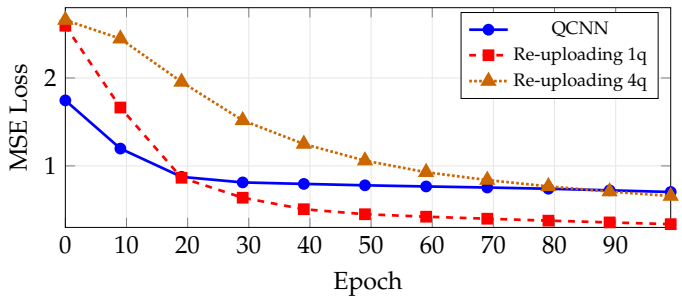


Figure 6.1: Training loss curves for quantum models on the standard 3-class setting (100 epochs, 500 training samples, seed 0). All three models converge, with the single-qubit re-uploading reaching the lowest loss (0.34).

All three quantum models show monotonically decreasing loss. The single-qubit re-uploading converges fastest, reaching a loss of 0.34 by epoch 99. The QCNN plateaus around epoch 30 and then decreases slowly to 0.70. The 4-qubit re-uploading model starts slowest but continues to improve throughout, reaching 0.66 by epoch 99.

The slower convergence of the 4-qubit model has three causes. First, its 72 parameters per classifier define a higher-dimensional loss landscape with more local minima. Second, the CZ entanglement gates create inter-qubit correlations that make individual parameter gradients less informative. Third, the averaged measurement $\langle H \rangle = \frac{1}{4}\sum_q Z_q$ dilutes the gradient signal compared to a single-qubit $\langle Z \rangle$. These effects are consistent with the barren plateau phenomenon [13].

6.5.1 Ablation: Effect of Dataset Size

I investigate whether the quantum models benefit from additional training data by varying the number of training samples from 100 to 1000 (all with 100 epochs, seed 42). For datasets larger than 500 samples, I use mini-batch training (batch size 100). Table 6.5 reports the resulting 3-class accuracy for each size.

Model	$n = 100$	$n = 250$	$n = 500$	$n = 1000$
Re-uploading 1q	0.750	0.730	0.730	0.730
Re-uploading 4q	0.695	0.700	0.680	0.700
QCNN (8q)	0.705	0.695	0.680	0.680
SVM (RBF)		0.970		
Random Forest		0.990		

Table 6.5: Standard 3-class accuracy vs. training set size (100 epochs, seed 42). All quantum models remain in the 68–75% range regardless of dataset size, confirming that circuit capacity—not data—is the bottleneck.

All three quantum models are remarkably stable across dataset sizes (68–75%), indicating that circuit capacity saturates with as few as 100 samples. No quantum model approaches the classical baselines. These results confirm that *circuit capacity*, not training data, is the primary bottleneck for quantum classifiers on this task.

6.6 Discussion

The experiments yield three complementary findings: (i) classical baselines dominate on the standard task (99% vs. 68–75% for quantum, regardless of training budget), consistent with the expectation that quantum advantage is unlikely on classically separable data [10, 20]; (ii) on zero-day, supervised classical and supervised re-uploading both fail ($< 3\%$ and 7–9%), while two approaches—the unsupervised autoencoder (70.5% $\pm$ 14.4%, trained on normal traffic only) and the supervised QCNN excl. barren seeds (68.6% $\pm$ 6.8%, trained on normal-vs-DoS but transferring to injection)—succeed at comparable levels through very different mechanisms (reconstruction error vs. quantum feature separation); and (iii) the QCNN has lower variance when it trains (6.8% vs. 14.4%) but collapses 3/10 seeds vs. 1/10 for the autoencoder, and the $56\times$ training-set asymmetry favours the autoencoder further. The QCNN's hierarchical pooling ($8 \to 1$) is what differentiates it from the re-uploading variants: same angle encoding, but multi-scale compression. The QCNN does not strictly dominate the autoencoder, but reaches comparable performance with $56\times$ less training data—consistent with literature on quantum feature spaces and out-of-distribution generalization [21, 11]. Remaining limitations and perspectives are discussed in Chapter 7.

Chapter 7

Conclusion

Summary

This work compared quantum machine learning classifiers with classical methods for IoT network intrusion detection, focusing on zero-day attacks. I evaluated five classical models (SVM, Random Forest, two neural networks, and an autoencoder anomaly detector) and three quantum architectures (QCNN, single- and 4-qubit data re-uploading) on ToN_IoT, using a one-vs-all strategy over three classes (normal, DoS, injection). All quantum and autoencoder results are averaged over 10 seeds.

On the standard 3-class task, classical supervised models reached near-perfect accuracy (99%) while quantum models plateaued at 68–75% regardless of training budget. An ablation confirmed that *circuit capacity*—not data—is the bottleneck: accuracy is nearly identical between 100 and 1000 training samples.

On the zero-day task, the picture inverted. Every supervised classical model failed ($< 3\%$), as did both re-uploading variants (7–9%). Two approaches reached $\sim$70%: the autoencoder at $70.5\% \pm 14.4\%$ (trained unsupervised on normal traffic only), and the QCNN at $68.6\% \pm 6.8\%$ on the 7 seeds that converged ($49.0\% \pm 30.5\%$ across all 10, with 3 barren-plateau collapses). The QCNN is trained *supervised* on normal-vs-DoS labels but transfers to injection, suggesting its feature map captures a broader notion of "not normal" rather than memorizing the DoS class. The QCNN does not strictly dominate, but reaches comparable accuracy with $56\times$ less training data and lower variance when convergent, at the cost of a 30% training-failure rate (vs. 10% for the autoencoder).

Limitations

All quantum experiments use a noiseless simulator (`lightning.qubit`), so real NISQ noise and decoherence are absent. QCNN zero-day accuracy varied from 0% to 76% across the 10 seeds, with 3 barren-plateau collapses—a single-seed run would have been misleading. The quantum models train on 500 samples and test on 200, while the autoencoder uses all $\sim$28,000 normal samples and the full 7,034 injection test set; this $56\times$ asymmetry favours the autoencoder, so the two means are not a strict head-to-head. Finally, training used Adam with adjoint differentiation (mathematically equivalent to parameter-shift), without the advanced techniques (warm restarts, layer-wise pre-training, smart initialization) proposed in the literature to mitigate barren plateaus.

Perspectives

Several directions extend this work:

- **More seeds.** 30–50 seeds would tighten the confidence interval on the failure rate and enable significance testing against the autoencoder.
- **Barren-plateau mitigation.** Layer-wise pre-training, identity-block init, or warm-start strategies could reduce the 30% failure rate—the single largest practical obstacle.
- **Noise simulation.** Realistic noise models (depolarizing, readout) would test whether the zero-day advantage survives under hardware conditions.
- **Larger circuits.** More qubits and depth could raise the capacity ceiling observed on the standard task.
- **Alternative encodings.** Amplitude or kernel-based encodings [21] would test whether the generalization is specific to angle encoding.
- **Real quantum hardware.** Execution on IBM Quantum or IonQ would ground-truth the simulator results.
- **Broader attack diversity.** Extending zero-day to backdoor, ransomware, or scanning attacks would test the generality of the anomaly-detection capability.

Despite classical methods' clear dominance on standard classification, the zero-day result offers an encouraging signal: both the QCNN and a classical autoencoder detect $\sim$70% of unseen attacks where every supervised classical model fails below 3%. The QCNN does so with 56$\times$ less training data and lower variance when convergent, but pays a 30% barren-plateau failure rate the autoencoder does not. The honest take-away is not "quantum wins," but that quantum hierarchical feature maps and classical reconstruction-based detectors are two complementary routes to the same kind of out-of-distribution generalization—and the quantum route is not (yet) clearly preferable. If barren plateaus can be mitigated and the effect confirmed on noisy hardware and broader attack taxonomies, the quantum approach may become attractive where labelled normal traffic is scarce.

Note on Writing Assistance

Parts of the text in this report were refined with the assistance of a large language model (LLM) for language formulation and proofreading. All scientific content—including the problem formulation, dataset preprocessing, model design, experiments, analyses, figures, and conclusions—is my own.

Appendix A

Complete Experimental Results

Table A.1 lists the 19 features retained from the ToN_IoT dataset after removing constant and identifier columns.

Feature	Type	Description
src_port, dst_port	Numerical	Source and destination transport port numbers
proto	Binary	Transport protocol (TCP = 0, UDP = 1)
service	Categorical	Application service (none, DNS, HTTP, SSL)
duration	Numerical	Flow duration in seconds
src_bytes, dst_bytes	Numerical	Bytes sent by source / destination
conn_state	Categorical	TCP connection state (REJ, S0, SF, etc.)
src_pkts, dst_pkts	Numerical	Packets sent by source / destination
src_ip_bytes, dst_ip_bytes	Numerical	IP-level bytes (including headers)
dns_qclass, dns_qtype, dns_rcode	Numerical	DNS query class, type, and response code
dns_AA, dns_RD, dns_RA, dns_rejected	Binary	DNS flags (authoritative, recursion, etc.)

Table A.1: The 19 features retained after removing constant and identifier columns.

Table A.2 consolidates all experimental results across models, tasks, and dataset sizes — including the autoencoder for completeness. Classical supervised models use the full balanced training set. The autoencoder uses the full ∼28,000 normal samples and is averaged over 10 seeds. Quantum results at $n = 500$ are mean ± std over 10 seeds; other quantum sizes use seed 42 only.

The complete per-seed breakdown for the QCNN over 10 seeds is given in Table 6.4 of Section 6.4. Re-uploading per-seed values are tightly clustered around their means (standard deviations of 0.039 and 0.011 on the standard task; 0.036 and 0.023 on zero-day) and are not reported individually.

A.1 Confusion Matrices

Table A.3 shows confusion matrices for the best classical model (Random Forest) and the best quantum model (QCNN, seed 0) on the standard 3-class task. Table A.4 shows the zero-day confusion matrices.

Model	**Task**	$n = 100$	$n = 250$	$n = 500$	$n = 1000$
Classical supervised — full training set (accuracy)					
Random Forest	Standard			0.990	
SVM (RBF)	Standard			0.970	
NN-Medium	Standard			0.960	
NN-Small	Standard			0.925	
NN-Small	Zero-day			0.028	
SVM (RBF)	Zero-day			0.024	
NN-Medium	Zero-day			0.017	
Random Forest	Zero-day			0.003	
Classical unsupervised — Autoencoder, ∼28,000 normal samples, 10 seeds (detection rate)					
Autoencoder	Zero-day			**0.705 ± 0.144**	
Quantum — Standard task, training subsamples of size n (accuracy)					
Re-uploading 1q	Standard	0.750	0.730	0.752 ± 0.039	0.730
Re-uploading 4q	Standard	0.695	0.700	0.692 ± 0.011	0.700
QCNN (8q)	Standard	0.705	0.695	0.679 ± 0.025	0.680
Quantum — Zero-day task, training subsamples of size n (detection rate)					
QCNN (8q)	Zero-day	0.005	0.020	**0.490 ± 0.305**	0.010
Re-uploading 1q	Zero-day	0.060	0.170	0.092 ± 0.036	0.105
Re-uploading 4q	Zero-day	0.040	0.040	0.075 ± 0.023	0.080

Table A.2: Consolidated results across all models, tasks, and training-set sizes. **Classical supervised** models are trained on the full balanced training set and reported as single numbers. **The autoencoder** uses all ∼28,000 normal samples, is tested on the full 7,034-sample injection set, and is averaged over 10 seeds. **Quantum** models use subsamples of size n (stratified for standard, fixed random for zero-day); $n = 500$ results are mean ± std over 10 seeds, other sizes use seed 42 only. The QCNN zero-day values at $n \neq 500$ use seed 42, which lands in a barren plateau — the multi-seed run at $n = 500$ shows that 7/10 seeds actually achieve 54–76% detection (mean 68.6% ± 6.8% excluding the 3 collapsed seeds).

A.2 Initialization Sensitivity

Figure A.1 visualizes the effect of parameter initialization across 10 random seeds for each quantum model. The QCNN zero-day task shows extreme variance, with three seeds collapsing to near-random performance (barren plateau), while the standard task and re-uploading models remain stable.

A.3 Quantum Circuit Architectures

Figure A.2 illustrates the QCNN architecture used in this work. The circuit operates on 8 qubits with 3 convolutional-pooling stages, progressively reducing the number of active qubits from 8 to 1.

Figure A.3 illustrates the single-qubit data re-uploading circuit.

Figure A.4 shows the multi-qubit variant used in this work: 4 qubits with 3 re-uploading layers and nearest-neighbor CZ entanglement between layers.

Random Forest — Accuracy 0.990			
	pred. DoS	**pred. Inj.**	**pred. Norm.**
DoS	65	0	1
Inj.	0	67	0
Norm.	0	1	66
QCNN (seed 0) — Accuracy 0.720			
	pred. DoS	**pred. Inj.**	**pred. Norm.**
DoS	64	0	2
Inj.	0	64	3
Norm.	43	8	16

Table A.3: Standard 3-class confusion matrices. The Random Forest achieves near-perfect separation. The QCNN correctly classifies DoS and injection but struggles with normal traffic, misclassifying 43 out of 67 normal samples as DoS.

QCNN (seed 0) — Zero-day Acc. 0.755		
	pred. Normal	**pred. Attack**
Injection	49	151

Random Forest — Zero-day Acc. 0.003		
	pred. Normal	**pred. Attack**
Injection	7010	24

Table A.4: Zero-day confusion matrices. The QCNN (seed 0) correctly detects 151 out of 200 unseen injection samples as attacks (75.5%). The Random Forest detects only 24 out of 7,034 (0.3%), classifying nearly all unseen attacks as normal.

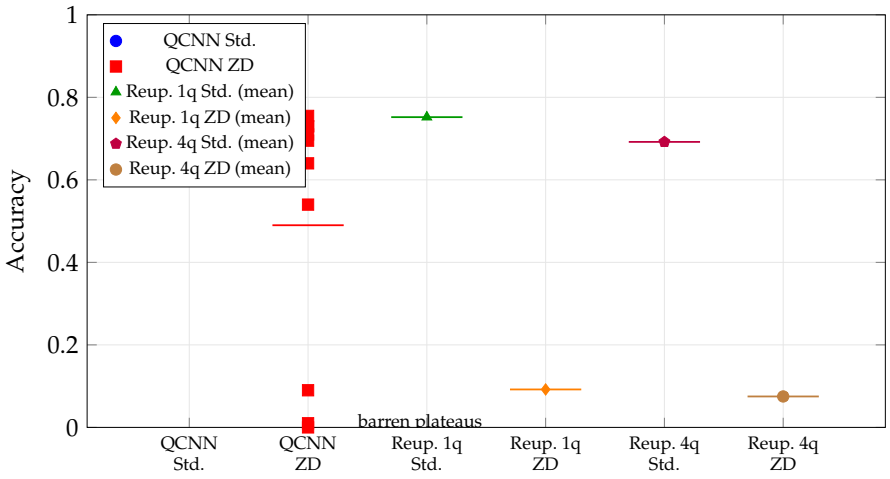


Figure A.1: Accuracy distribution across 10 random initialization seeds for each quantum model and task (500 samples, 100 epochs). QCNN points are individual seeds; re-uploading is shown as a single mean marker because its per-seed values are tightly clustered (standard deviations 0.011–0.039). The QCNN zero-day task exhibits three barren-plateau collapses at $\leq 9\%$, while seven seeds achieve 54–76%. All other model–task combinations are stable across seeds.

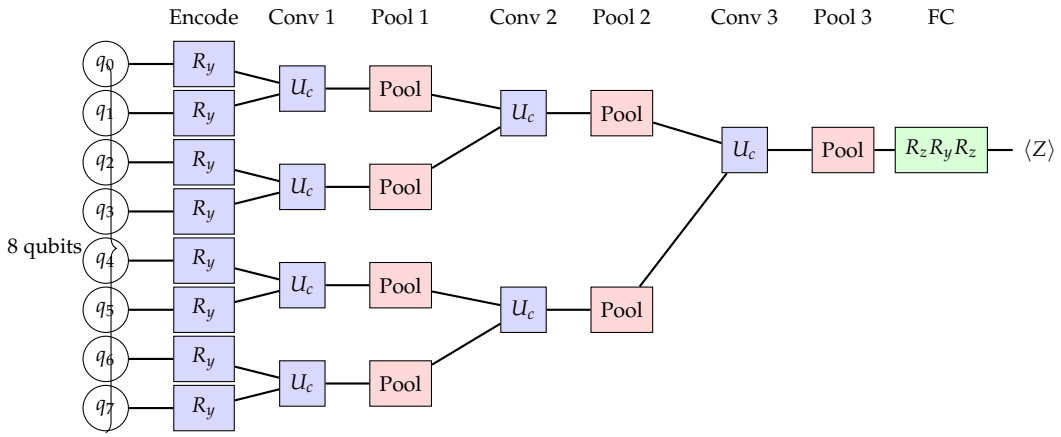


Figure A.2: QCNN architecture ($8 \to 4 \to 2 \to 1$ qubits). Input features are encoded via R_y rotations. Three convolutional-pooling stages hierarchically reduce the circuit width. Each convolutional unitary U_c applies 6 parameterized rotations; each pooling layer uses CRZ–CRY (2 parameters) to transfer information before discarding one qubit. A final $R_z R_y R_z$ layer on the remaining qubit produces the output $\langle Z \rangle \in [-1, +1]$. Total: 27 parameters per binary classifier.

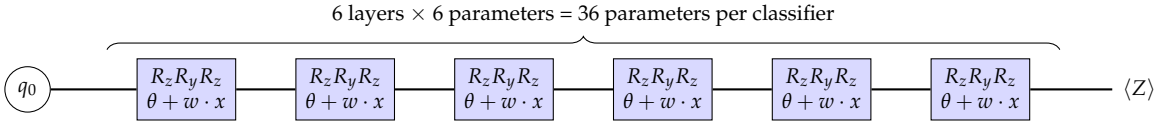


Figure A.3: Single-qubit data re-uploading circuit (6 layers). Each layer applies parameterized $R_z R_y R_z$ rotations where the rotation angles are $\theta_j^{(l)} + w_j^{(l)} \cdot x_{f(j,l)}$, combining trainable weights with input features that cycle across layers. The same qubit is reused across all layers, enabling a single qubit to learn complex decision boundaries. Total: 36 parameters per binary classifier.

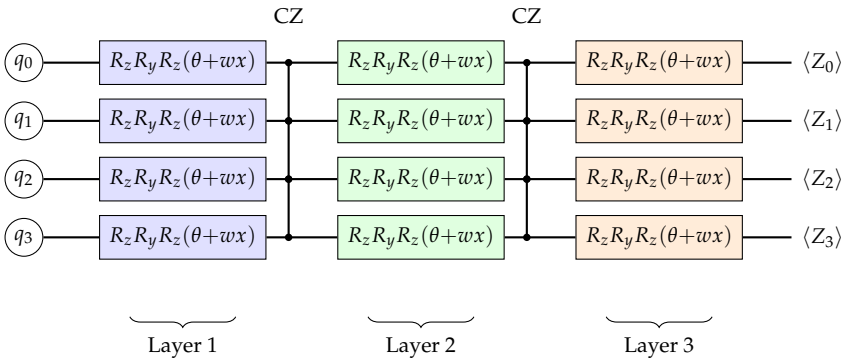


Figure A.4: Multi-qubit data re-uploading circuit (4 qubits, 3 layers). Each qubit independently applies parameterized $R_z R_y R_z$ rotations that mix input data $\mathbf{x}$ with trainable weights, with nearest-neighbor CZ gates entangling the qubits between layers. The final output is the average of the per-qubit $\langle Z \rangle$ measurements, expressed as a single Hamiltonian measurement $\langle H \rangle = \frac{1}{n} \sum_q Z_q$. Total: $6 \times 4 \times 3 = 72$ parameters per binary classifier.

Appendix B

Bibliography

[1] Marcello Benedetti, Erika Lloyd, Stefan Sack, and Mattia Fiorentini. Parameterized quantum circuits as machine learning models. *Quantum Science and Technology*, 4(4):043001, 2019. doi: 10.1088/2058-9565/ab4eb5.

[2] Adrián Pérez-Salinas, Alba Cervera-Lierta, Elies Gil-Fuster, and José I. Latorre. Data re-uploading for a universal quantum classifier. *Quantum*, 4:226, 2020. doi: 10.22331/q-2020-02-06-226.

[3] Tak Hur, Leeseok Kim, and Daniel K. Park. Quantum convolutional neural network for classical data classification. *Quantum Machine Intelligence*, 4(1):3, 2022. doi: 10.1007/s42484-021-00061-x.

[4] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018. doi: 10.22331/q-2018-08-06-79.

[5] UNSW Cyber Range Lab. ToN_IoT datasets. https://research.unsw.edu.au/projects/toniot-datasets, 2020. Accessed: February 2026.

[6] Xanadu. PennyLane: Automatic differentiation of hybrid quantum-classical computations. https://pennylane.ai/, 2024.

[7] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition, 2010.

[8] Marco Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3:625–644, 2021. doi: 10.1038/s42254-021-00348-9.

[9] Ryan LaRose and Brian Coyle. Robust data encodings for quantum classifiers. *Physical Review A*, 102(3):032420, 2020. doi: 10.1103/PhysRevA.102.032420.

[10] Maria Schuld, Ryan Sweke, and Johannes Jakob Meyer. Effect of data encoding on the expressive power of variational quantum machine learning models. *Physical Review A*, 103(3):032430, 2021. doi: 10.1103/PhysRevA.103.032430.

[11] Maria Schuld, Alex Bocharov, Krysta M. Svore, and Nathan Wiebe. Circuit-centric quantum classifiers. *Physical Review A*, 101(3):032308, 2020. doi: 10.1103/PhysRevA.101.032308.

[12] Kosuke Mitarai, Makoto Negoro, Masahiro Kitagawa, and Keisuke Fujii. Quantum circuit learning. *Physical Review A*, 98(3):032309, 2018. doi: 10.1103/PhysRevA.98.032309.

[13] Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9(1):4812, 2018. doi: 10.1038/s41467-018-07090-4.

[14] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980.

[15] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023. doi: 10.1016/j.patter.2023.100804.

[16] Ashkan Motamedifar. Quantum computing for machine learning — IoT network intrusion detection. https://github.com/ashkan-motamedifar/quantum-ml-ter, 2026. Source code repository.

[17] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[18] Davide Chicco and Giuseppe Jurman. The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21(1):6, 2020. doi: 10.1186/s12864-019-6413-7.

[19] Nour Moustafa. A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets. *Sustainable Cities and Society*, 72:102994, 2021. doi: 10.1016/j.scs.2021.102994.

[20] Joseph Bowles, Shahnawaz Ahmed, and Maria Schuld. Better than classical? the subtle art of benchmarking quantum machine learning models. *arXiv preprint arXiv:2403.07059*, 2024.

[21] Vojtěch Havlíček, Antonio D. Córcoles, Kristan Temme, Aram W. Harrow, Abhinav Kandala, Jerry M. Chow, and Jay M. Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567:209–212, 2019. doi: 10.1038/s41586-019-0980-2.